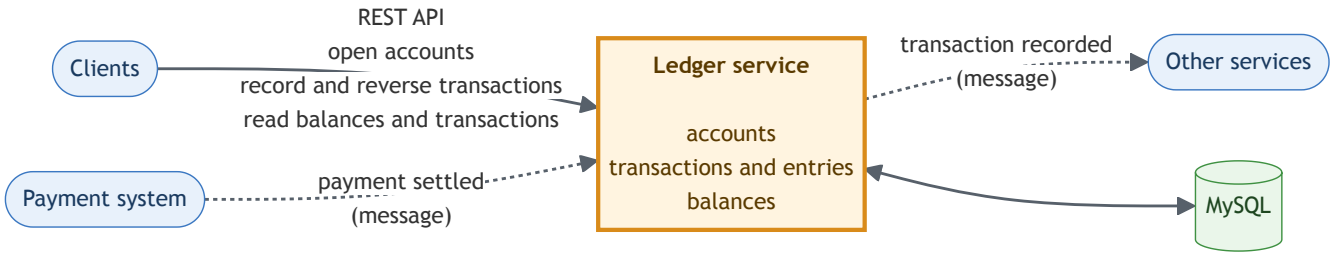
# Backend Code Challenge — Ledger API

## What it's about

Build a small **double-entry ledger**: the service a payments platform uses to record where money is and how it moved.

Money is held in **accounts**. It moves only through **transactions**. A transaction is two or more **entries** (debits and credits) that always balance. Once a transaction is recorded, it is never changed: a mistake is corrected by a reversal, which is a new transaction.




Transactions are recorded in two ways: directly, through the REST API, and automatically, when a payment-settled message arrives from the payment system. Every recorded transaction is announced to the other services. Clients retry, messages are redelivered and requests race each other, yet every balance must stay exact.

The design is yours: the resources, URLs, request and response shapes, headers, errors and versioning of the API, and the message broker. We describe below what the ledger holds and how it must behave.

You're free to use AI tools however you like, or not at all.

If something isn't specified, make a reasonable call, note it in the README and move on.

## The model

**Account**

- An identifier
- A currency, as its ISO 4217 code. The ledger supports EUR, USD and JPY, with the decimal places ISO 4217 gives them: 2, 2 and 0
- An overdraft limit: how far below zero the balance may go. Zero unless set when the account is opened
- A balance. A **credit** increases it and a **debit** decreases it

**Transaction**

- An identifier
- A description
- Two or more entries
- When it was recorded
- For a reversal, the transaction it reverses

**Entry**

- The account it posts to
- A direction: debit or credit
- An amount: positive, with no more decimal places than its account's currency allows

## Examples

All accounts start at zero. *Settlement EUR* and *Settlement JPY* are the ledger's own accounts, with no overdraft limit. Every other account has an overdraft limit of zero.

**1. A payment of 125.50 EUR settles for Merchant 42** (from a payment-settled message)

| Account | Debit | Credit | Balance after |
|---|---|---|---|
| Settlement EUR | 125.50 | | −125.50 |
| Merchant 42 (EUR) | | 125.50 | 125.50 |

**2. Merchant 42 is paid out 98.50 EUR, with a fee of 1.50** (through the API)

| Account | Debit | Credit | Balance after |
|---|---|---|---|
| Merchant 42 (EUR) | 100.00 | | 25.50 |
| Payouts EUR | | 98.50 | 98.50 |
| Fees EUR | | 1.50 | 1.50 |

The EUR balances still add up to zero: −125.50 + 25.50 + 98.50 + 1.50.

**3. The payout is reversed**

A new transaction, linked to transaction 2, with every entry flipped:

| Account | Debit | Credit | Balance after |
|---|---|---|---|
| Merchant 42 (EUR) | | 100.00 | 125.50 |
| Payouts EUR | 98.50 | | 0.00 |
| Fees EUR | 1.50 | | 0.00 |

Transaction 2 stays exactly as it was. Reversing it a second time is refused.

**4. A payment of ¥15,000 settles for Merchant 7**

| Account | Debit | Credit | Balance after |
|---|---|---|---|
| Settlement JPY | 15000 | | −15000 |
| Merchant 7 (JPY) | | 15000 | 15000 |

**5. Refused**, starting from the balances after example 2. Nothing is recorded, and the caller is told why:

| Request | Why it is refused |
|---|---|
| Debit Merchant 42 by 100.00, credit Payouts EUR by 99.00 | Debits and credits don't balance |
| Debit Merchant 42 by 50.00, credit Payouts EUR by 50.00 | Merchant 42 would go to −24.50, past its overdraft limit of 0 |
| Debit Merchant 7 by ¥100.5, credit Payouts JPY by ¥100.5 | JPY has no decimal places |
| Debit Merchant 42 by 10.00 EUR, credit Merchant 7 by ¥10 | The accounts are in different currencies |
| One entry only, crediting Merchant 42 by 10.00 | A transaction needs at least two entries |
| An entry posting to an account that doesn't exist | Unknown account |
| Opening an account in `EURO` or `ABC` | Not an ISO 4217 currency code |
| Opening an account in `GBP` | A valid ISO 4217 code, but not a currency the ledger supports |

## Functional requirements

### 1. Accounts

- Open an account
- Read an account with its current balance

### 2. Recording a transaction

A transaction is recorded only if all of these hold:

- it has at least two entries, and every amount is positive and fits its currency's decimal places
- all of its accounts exist and are in the same currency
- its debits and its credits add up to the same total
- no account would go past its overdraft limit

It is recorded **all or nothing**: every entry or none. When it is refused, the caller can tell why and which part of the request was at fault.

- A recorded transaction can be read back
- Transactions and their entries are never updated or deleted

### 3. Reversing a transaction

- Reversing a transaction records a new transaction with every entry of the original flipped (debit ↔ credit), linked to the original
- A transaction can be reversed only once. A reversal itself can't be reversed

- A reversal follows the same rules as any other transaction, overdraft limits included

### 4. Recording settled payments

The payment system sends a **payment-settled** message through a message broker. It carries an event id, a payment id, the merchant's account id, an amount and a currency.

- Each message records a transaction that debits the settlement account in that currency and credits the merchant's account, as in examples 1 and 4. The settlement accounts always exist
- A message that can't be recorded (malformed, an unknown account, the wrong currency) must not block the messages behind it, and must not be lost without trace

Provide a way to send sample payment-settled messages.

### 5. Announcing recorded transactions

Every recorded transaction, reversals included, is announced to the other services, with its details. How you do it is your choice. In the README, say what a consumer can rely on: can it miss a transaction, can it see one twice, in what order do transactions arrive, and what happens if the service crashes right after recording one?

### 6. Behaving correctly under pressure

The ledger runs on several instances. Each of these must hold however the requests and messages are interleaved:

- **Concurrency:** transactions touching the same accounts at the same time never break an overdraft limit, never lose an update, and don't deadlock. This includes money moving A → B and B → A at the same moment
- **Idempotency:** a client that doesn't get an answer can safely send the same request again, whether it opens an account, records a transaction or reverses one. The request takes effect at most once, and the retry gets the same answer the first attempt would have given, even if it lands on another instance or the first attempt crashed halfway. The ledger never mistakes a different request for a retry
- **Redelivery:** the same payment is recorded once, however many times it arrives, including when the payment system re-sends it under a new event id
- **Integrity:** at any moment, the balances of all accounts in one currency add up to zero

## Non-functional requirements

- **Stack:** Java 17, Spring Boot 4, MySQL, REST, and a message broker of your choice
- **Running:** everything runs in Docker. One script builds the service, starts it with everything it needs, and runs the tests. We should need nothing installed but Docker, Git and a JDK
- **Design:** structure the code as you see fit, and explain your choice in the README
- **Money is exact:** amounts are never approximated, anywhere: 0.10 + 0.20 is always exactly 0.30
- **Tests** that prove the guarantees in section 6, and that a refused transaction leaves nothing behind, against the real database and broker

## Nice to have

- **API first:** the API contract designed and written down before the code, and kept in step with it
- An account's entries, newest first, paged, each with the balance after it
- The balance of an account as at a past instant
- Being able to follow one request or one payment through the logs and through the announcements it caused

## Improvements

End the README with an **Improvements** section: what you would change before production, and what you skipped and why.

## Deliverables

A public Git repository (GitHub, GitLab or similar) with its commit history, containing:

- **The code and tests**
- **`README.md`:** how to run the script; sample requests, and how to send a sample payment-settled message; your design decisions, including the API, the broker and how transactions are announced; Improvements
- **A few words on AI**, in the README or in `AI_USAGE.md`: whether you used it, for what, and how you checked what it produced. Commit any instruction files you gave it

## How we evaluate

- Ledger integrity: balanced, atomic, immutable
- Idempotency, and correctness under concurrency, retries and redelivery
- Event-driven design: consuming and announcing reliably
- API design: resources, errors, retries, how it can evolve
- Domain model and code clarity
- Tests that prove the guarantees
- Your decisions, and how clearly you explain them